

Paralelização na camada de aplicação - Unidade 1

Eduardo Borges · Luiz Henrique Rocha · Eliziane Mota · Pedro Iranildo · Ricardo Severiano de Souza Filho

Série publicada em 06/10/2026: 40 medições de resposta HTTP. Fonte: [bench/measurements.csv](#).

1. Justificativa

Operação escolhida

A detecção fuzzy de duplicatas compara nomes textuais de medicamentos e conta pares cuja distância de Levenshtein é menor ou igual a dois. A demonstração usa dados sintéticos reprodutíveis de `DataGeneratorService`.

Entre candidatas como relatórios, cruzamento de dados, ordenação e validações em lote, esta operação oferece trabalho de CPU explícito e independente por par. Em `DuplicateDetectionService`, os textos já estão em memória e o laço de comparação não acessa banco nem rede. A escolha se apoia nesse código e no custo medido, sem generalizar o gargalo das outras operações, que dependeria de suas implementações e cargas.

Custo e evidência

A versão sequencial percorre i de zero a $n-1$ e j de $i+1$ a $n-1$. Cada par aparece uma vez: são $n(n-1)/2$ comparações. `FuzzyMatcher` calcula Levenshtein por programação dinâmica, com custo $O(L^2)$ para textos de comprimento limitado por L . O trabalho total é $O(n^2 \cdot L^2)$, ou $O(n^2)$ quando L permanece limitado.

Na série atual, a média HTTP sequencial é 23.883,00 ms para 8.000 registros e 97.867,00 ms para 16.000: crescimento de aproximadamente 4,10x. Isso é compatível com o custo quadrático, sem provar a complexidade apenas por dois pontos. O endpoint inclui também geração de dados e transporte HTTP; a identificação de trabalho de CPU vem do laço e do cálculo de Levenshtein. Os dados completos e os limites de extrapolação constam em medições ([medicoes.md](#)).

Divisão e agregação

`runPartitioned` divide o intervalo de índices externos i em fatias disjuntas, usando `ceil(n / slices)`. Cada tarefa compara seus índices com todos os $j > i$, sem perder ou repetir pares. A lista é apenas lida durante a detecção.

Cada tarefa mantém seu próprio `localDuplicates` e retorna uma contagem. `invokeAll` aguarda as tarefas; a thread chamadora soma os valores obtidos com `Future.get()`. Não há contador compartilhado atualizado no laço, portanto essa organização dispensa locks ou operações atômicas para contar pares.

As fatias têm quantidades semelhantes de índices, mas cargas diferentes: os primeiros índices possuem mais parceiros. Esse desbalanceamento e o custo do pool e da agregação limitam o ganho. Com oito threads, a série atual registra speedups de 3,31x e 2,37x, respectivamente.

As 40 execuções retornam contagens iguais para cada tamanho. `DuplicateDetectionServiceTest` verifica a equivalência entre versões e casos de borda com testes automatizados. A análise ([analise.md](#)) relaciona o resultado a concorrência, paralelismo e possibilidades futuras da Unidade 2.

2. Serviço e versões

A requisição chega ao endpoint Spring Boot, os dados são gerados, a camada de aplicação detecta os pares e retorna um JSON com `duplicatesFound`, `processingTimeMs`, `dataGenerationTimeMs` e `status`.

GET /api/duplicates/detect?size=8000&threads=8&mode=platform

Classe	Responsabilidade no código
DuplicateController	Valida size, threads e mode; seleciona a versão sequencial para platform com uma thread e a versão paralela para mais threads.
DataGeneratorService	Gera nomes sintéticos de medicamentos com seed 42 e possíveis erros de digitação.
DuplicateDetectionService	Executa detectSequential, detectParallel e runPartitioned; soma contagens após Future.get().
FuzzyMatcher	Calcula a distância de Levenshtein usando duas linhas de programação dinâmica.

Sequencial e threads de plataforma

`detectSequential` percorre cada par $i < j$ uma vez. `detectParallel` cria `Executors.newFixedThreadPool(t)`. `runPartitioned` divide os índices externos em t fatias contíguas de tamanho $\text{ceil}(n/t)$. Cada `Callable` lê a lista e atualiza apenas seu contador local; `invokeAll` aguarda as tarefas e a thread chamadora agrega os resultados.

A validação limita size a 20.000 e threads a quatro vezes os processadores disponíveis à JVM. O serviço também oferece virtual threads como opção; esta série mede apenas platform, com 1, 2, 4 e 8 threads.

Equivalência funcional da série atual

n	Sequencial	2 threads	4 threads	8 threads
8000	3198218	3198218	3198218	3198218
16000	12801507	12801507	12801507	12801507

Cada célula corresponde à contagem repetida nas cinco execuções da configuração. Todas as 40 observações têm status SUCCESS. Os testes JUnit de `DuplicateDetectionServiceTest` verificam equivalência e casos de borda.

3. Medições

Série única publicada em **06/10/2026**, baseada em dados brutos (../bench/measurements.csv). São 40 execuções: tamanhos 8.000 e 16.000, modo platform, 1, 2, 4 e 8 threads, cinco repetições por configuração.

Método e evidências

O endpoint é `GET /api/duplicates/detect?size={n}&threads={t}&mode=platform`. Com uma thread, o controlador chama a versão sequencial; com mais threads, usa um pool fixo. `DataGeneratorService` gera nomes sintéticos de medicamentos com seed 42 e possíveis erros de digitação.

O script `measure.ps1` (../bench/measure.ps1) prevê uma execução de aquecimento com 8.000 registros e uma thread, excluída do CSV. Isso reduz efeitos de inicialização, mas não garante que todos os efeitos da JVM sejam eliminados. O projeto requer Java 21; o CSV não registra hardware, sistema operacional nem versão exata do JDK da coleta.

`responseTimeMs` é o tempo de parede medido pelo cliente com `Stopwatch` ao redor de `Invoke-RestMethod`. Inclui chamada HTTP, processamento no servidor e retorno ao cliente. `processingTimeMs` mede internamente apenas a detecção; a tabela usa exclusivamente `responseTimeMs`.

Para cada configuração, $T_t = \text{soma}(\text{responseTimeMs das cinco repetições}) / 5$. O speedup é T_1 / T_t , com a mesma entrada. Os valores são arredondados a duas casas após o cálculo.

Todas as 40 linhas registram SUCCESS. Para 8.000 registros, todas retornam 3.198.218 pares duplicados; para 16.000, 12.801.507. A igualdade sustenta a equivalência nesta série, sem constituir prova geral de ausência de falhas de concorrência. `DuplicateDetectionServiceTest` contém testes automatizados de equivalência entre versões e casos de borda.

Resultados atuais

n	threads	modo	média HTTP (ms)	speedup
8000	1	platform	23883.00	1.00x
8000	2	platform	18925.60	1.26x
8000	4	platform	11105.20	2.15x
8000	8	platform	7209.80	3.31x
16000	1	platform	97867.00	1.00x
16000	2	platform	81400.00	1.20x
16000	4	platform	69518.20	1.41x
16000	8	platform	41297.40	2.37x

O tempo sequencial cresce $97867 / 23883 \approx 4,10x$ ao dobrar a entrada. Esse comportamento é compatível com o custo quadrático de pares, mantendo textos de tamanho semelhante; dois tamanhos não provam empiricamente a complexidade.

Há variação entre repetições, especialmente em 16.000 registros e duas threads: de 72.741 a 102.051 ms. A média inclui todas as cinco observações. Os dados não permitem atribuir essa variação a utilização de CPU, temperatura ou outros processos. Nenhuma virtual thread foi medida nesta série.

Escala do enunciado e limites

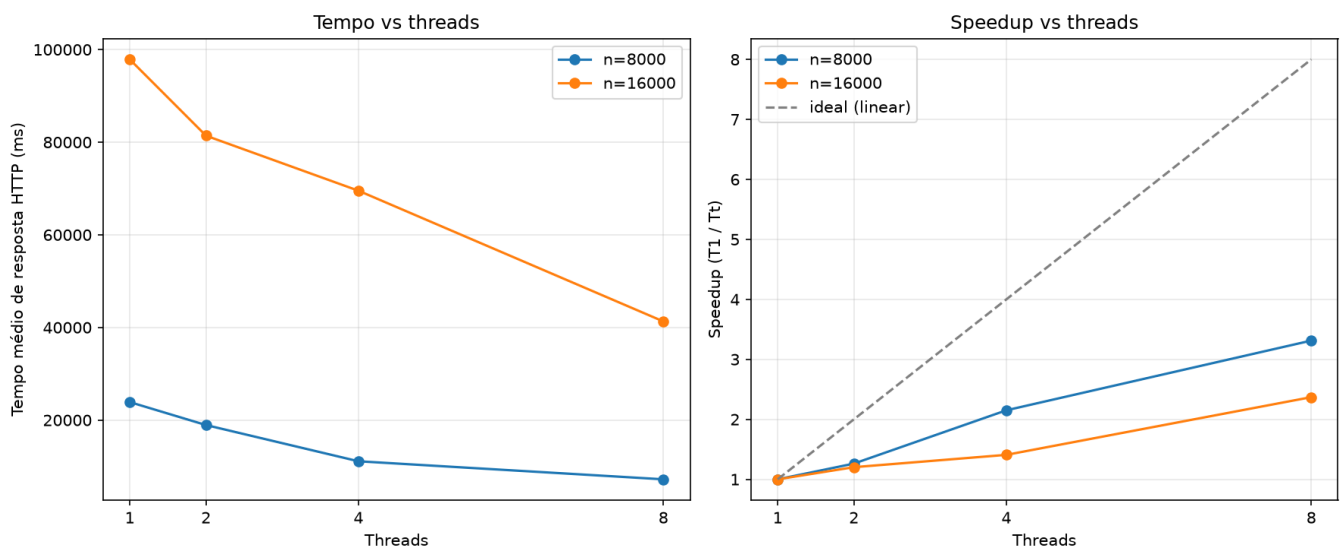
100 mil e 1 milhão de registros são exemplos de escala do enunciado, sem execuções correspondentes no CSV. O endpoint atual limita size a 20.000 e threads a quatro vezes os processadores disponíveis à JVM.

Uma estimativa ilustrativa, mantendo o mesmo custo por par, aplica $T(n) = 23883 \times (n / 8000)^2$ ms à média sequencial atual:

n	tempo sequencial estimado, sem execução
100.000	$\approx 3.731,72$ s (1,04 h)
1.000.000	$\approx 373.171,88$ s (4,32 dias)

Esses números dependem de textos, recursos e condições semelhantes. Não são previsões validadas; não se extrapola um speedup constante, pois ele já varia entre os dois tamanhos medidos. A justificativa (justificativa.md) explica a Big-O; a análise (analise.md) discute limites e propostas futuras.

Gráfico e reprodução



O gráfico em docs/speedup.png é uma cópia do gráfico de benchmark (../bench/speedup.png). O script plot_speedup.py (../bench/plot_speedup.py) recalcula o CSV agregado (../bench/measurements_agg.csv) e os gráficos a partir da série bruta, sem executar nova coleta:

```
python bench/plot_speedup.py
```

Para validar os dados sem gerar arquivos nem carregar matplotlib, e executar os testes do script:

```
python bench/plot_speedup.py --check
python -m unittest discover -s bench -p "test_*.py"
```

Para gerar o relatório PDF a partir dos Markdown e do CSV agregado:

```
python bench/build_report.py
```

Execute na raiz do projeto com Python 3.12+. Para gerar os gráficos e o PDF, instale as dependências fixadas em bench/requirements.txt: matplotlib==3.11.2 e reportlab==4.4.9.

```
python -m pip install -r bench/requirements.txt
```

O README (../README.md#benchmark-de-parallelismo-threads) inclui instalação em ambiente isolado. O relatório PDF (RotaVital_Threads_Relatorio.pdf) reúne a entrega documental.

4. Análise - 18 linhas

Média HTTP: cinco repetições; 1, 2, 4 e 8 threads de plataforma.
Com oito threads: 3,31x em 8.000 registros e 2,37x em 16.000.
O ganho é não linear e fica abaixo do ideal de 8x.
Em 16.000 e duas threads: 72.741–102.051 ms; a causa não foi isolada.
Cada índice i faz $n-i-1$ comparações; fatias iguais ficam desbalanceadas.
Criar o pool, agendar e agregar tarefas acrescenta overhead.
Contadores locais são somados via `Future.get()`, sem contador compartilhado.
Os mesmos $n(n-1)/2$ pares são comparados nas duas versões.
Levenshtein leva o trabalho a $O(n^2 \cdot L^2)$, com textos limitados por L .
Para L fixo, o custo é $O(n^2)$; dobrar n elevou o tempo sequencial 4,10x.
Threads reduzem o tempo de parede e mantêm a ordem de complexidade.
Na Mesa DJ, concorrência intercala eventos de teclado, MIDI e áudio.
Aqui, paralelismo executa fatias juntas quando há núcleos disponíveis.
Uma proposta futura é balancear tarefas pela quantidade de pares.
Selecionar candidatos reduz comparações, com risco de falso negativo.
Execução assíncrona e distribuição entre máquinas são propostas para U2.
Essas propostas ainda não foram implementadas nem medidas.
100 mil e 1 milhão têm apenas estimativas de escala.

Fontes da entrega

`bench/measurements.csv`; `bench/measurements_agg.csv`; `bench/speedup.png`; `docs/justificativa.md`; `docs/medicoes.md`; `docs/analise.md`; código em `src/main/java/com/rotavital`; testes em `src/test/java/com/rotavital`.